



El siguiente taller consiste en la implementación y uso de la clase `Handle`, con la finalidad de obtener mejor eficiencia en ciertas operaciones sobre estructuras de datos enlazadas.

Link de entrega: Formulario Campus Virtual (**Adjuntar** los archivos `ABB.java`, `ListaEnlazada.java` y `SistemaPedidos.java`)

Última fecha de entrega: domingo 11/11 a las 23:59hs.

Consigna

Dada una empresa a la cual le llegan pedidos de distintos productor por distintos clientes, tenemos que armar un *Sistema de Pedidos* al cual registramos los pedidos entrantes, consultamos los existentes y descolamos según la prioridad con la que llegan o su vez, siendo el ID del pedido su prioridad.

En esta empresa llegan pocos pedidos nuevos, se consulta seguido el listado de pedidos y se los atienden más seguido por su prioridad que por su orden de llegada. Por estas razones, el diseño de nuestro sistema consistirá en dos secuencias de pedidos: una ordenada por orden de llegada y otra por prioridad. Pero para mejorar la eficiencia al desencolar los elementos, vamos a enlazar estas dos estructuras con *aliasing*.

En este taller, la implementación de la clase `ListaEnlazada` para viene modificada para que utilice y *emita handles*. Un `Handle` es un objeto que actúa como referencia a otro objeto, permitiendo acceder y modificar al referenciado de manera indirecta en $O(1)$, con el conocimiento de la instancia de la clase que *emitió* el `Handle`. En este caso, el handle será un objeto que contiene un puntero *privado* al nodo de la lista enlazada. De esta manera, podremos acceder, modificar y eliminar los elementos de la lista sin necesidad de recorrerla.

ara nuestro *Sistema de Pedidos* vamos a utilizar una `ListaEnlazada` para almacenar los pedidos en orden de llegada y un `ArrayList` para almacenar sus `Handle` en orden de ID.

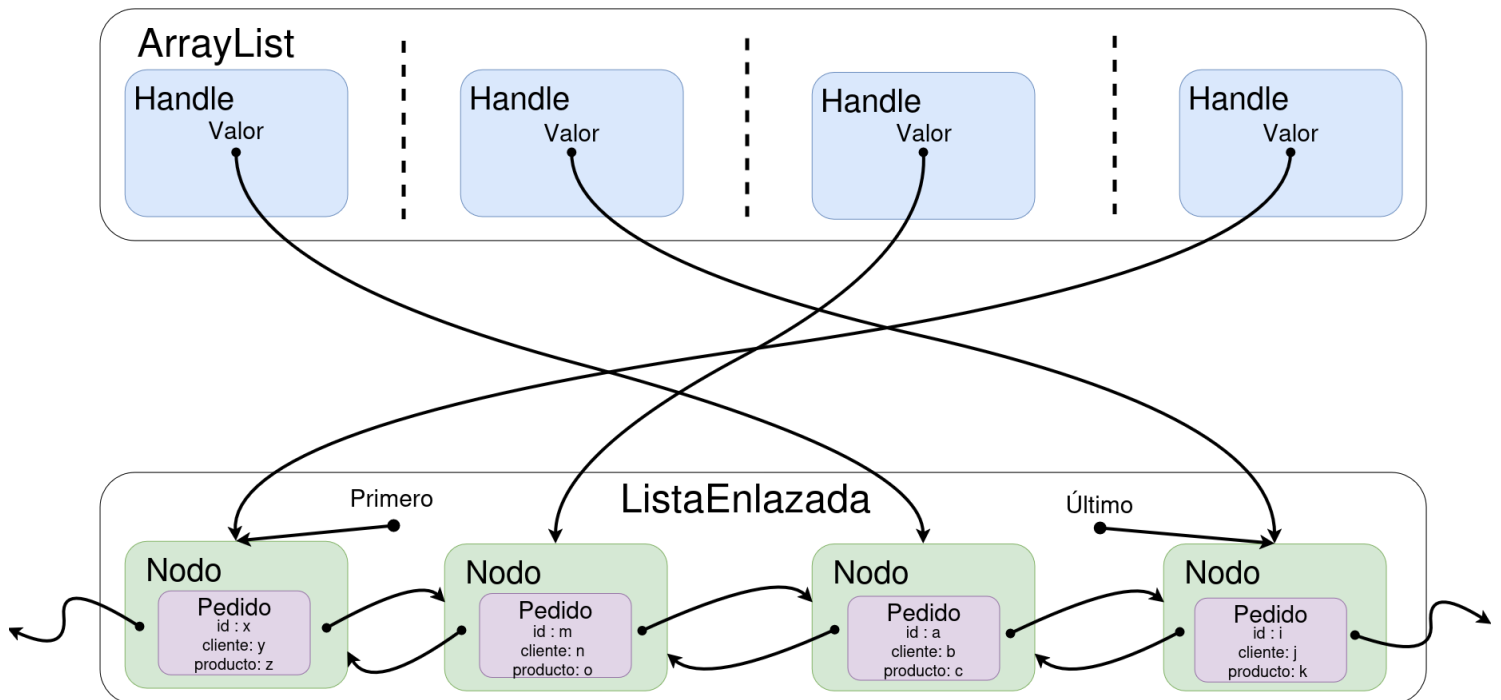


Figura 1: Diagrama de estructuras de datos enlazadas utilizando handles.

Para resolver el taller, deben modificar la clase `ListaEnlazada<T>` (dentro del archivo `ListaEnlazada.java`) de manera tal que devuelva un `Handle<T>` al agregar un elemento y permita eliminar elementos a través de él.

1. Definir la clase anidada de `ListaEnlazada`, `HandleLE` que implemente la interfaz `Handle`. Esta clase debe contener una referencia al nodo que contiene el elemento referenciado por el handle.
2. Implementar el método `T valor()` de la clase `HandleLE` para obtener el valor del nodo referenciado por el handle.
3. Implementar el método `void eliminar()` de la clase `HandleLE` para eliminar el nodo referenciado por el handle. Utilicen el método privado `eliminarNodo()` de `ListaEnlazada`. Pueden invocar la instancia relevante de la `ListaEnlazada` con `ListaEnlazada.this`
4. Implementar los métodos `int compareTo(Handle<T>)` y `String toString()`, que dejan la comparación y pasar a string a los valores almacenados.
5. Modificar los 3 métodos que agregan elementos nuevos a la lista para que devuelvan un `Handle` del elemento agregado.

Implementar la clase `SistemaPedidos` (en el archivo `SistemaPedidos.java`) de manera tal que permita agregar pedidos, obtener el pedido con menor identificador, quitar pedidos por orden de llegada o identificador y listarlos tanto por su orden de llegada como por su identificador. Recordar que los identificadores siguen una distribución aleatoria.

1. `void agregarPedido(Pedido pedido);`
Agrega un pedido al sistema con complejidad $O(n)$.
2. `Pedido proximoPedidoPorLlegada();`
Devuelve el próximo pedido por orden de llegada y lo elimina del sistema con complejidad $O(n)$.
3. `Pedido proximoPedidoPorId();`
Devuelve el próximo pedido por orden de id y lo elimina del sistema con complejidad $O(1)$. (Tener en cuenta que agregar y eliminar al final de un `ArrayList` es $O(1)$)
4. `Pedido pedidoMenorId();`
Devuelve el pedido con menor identificador con complejidad $O(1)$.
5. `String obtenerPedidosEnOrdenDeLlegada();`
Devuelve un string con los pedidos en orden de llegada en $O(n)$ (e.g., “[28, 71, 17, 261, 21]”).
6. `String obtenerPedidosOrdenadosPorId();`
Retorna un string con los pedidos ordenados por su identificador en $O(n)$ (e.g., “{17, 21, 28, 71, 261}”).